

PYTHON IMPORTANT TOPICS – DEFINITIONS & SYNTAX

1. VARIABLES

Definition: Used to store data values.

Syntax:

```
name = "Deepak"
```

```
age = 20
```

2. DATA TYPES

Definition: Different kinds of values used in Python.

Syntax:

```
x = 10 # int
```

```
y = 3.5 # float
```

```
s = "Hi" # string
```

```
b = True # boolean
```

3. TYPE CASTING

Definition: Convert one data type to another.

Syntax:

```
int("5"), str(10), float("3.14")
```

4. INPUT / OUTPUT

Definition: Take input and print output.

Syntax:

```
name = input("Enter: ")
```

```
print(name)
```

5. IF / ELIF / ELSE

Definition: Conditional decision making.

Syntax:

```
if x > 0:
```

```
    print("Positive")
```

```
elif x == 0:
```

```
    print("Zero")
```

else:

```
print("Negative")
```

6. LOOPS (FOR)

Definition: Repeat actions a set number of times.

Syntax:

```
for i in range(5):
```

```
    print(i)
```

7. LOOPS (WHILE)

Definition: Repeat until a condition becomes false.

Syntax:

```
while x < 5:
```

```
    x += 1
```

8. BREAK & CONTINUE

Definition: Control loop flow.

Syntax:

```
break # stops loop
```

```
continue # skips to next iteration
```

9. LIST

Definition: Ordered, changeable collection.

Syntax:

```
lst = [1,2,3]
```

```
lst.append(4)
```

10. TUPLE

Definition: Ordered, unchangeable collection.

Syntax:

```
t = (1,2,3)
```

11. SET

Definition: Unordered collection with unique items.

Syntax:

```
s = {1,2,3}
```

12. DICTIONARY

Definition: Key-value data store.

Syntax:

```
d = {"name":"Deepak", "age":21}
```

13. FUNCTIONS

Definition: Reusable block of code.

Syntax:

```
def add(a,b):
```

```
    return a+b
```

14. LAMBDA FUNCTION

Definition: Small anonymous one-line function.

Syntax:

```
square = lambda x: x*x
```

15. MODULE IMPORT

Definition: Reuse external code.

Syntax:

```
import math
```

16. EXCEPTION HANDLING

Definition: Handle runtime errors safely.

Syntax:

```
try:
```

```
    x = 1/0
```

```
except ZeroDivisionError:
```

```
    print("Error")
```

17. CLASS (OOP)

Definition: Blueprint for creating objects.

Syntax:

```
class Student:
```

```
def __init__(self,name):
```

```
self.name=name
```

18. FILE HANDLING

Definition: Read/write files.

Syntax:

```
with open("data.txt","r") as f:
```

```
print(f.read())
```

19. LIST COMPREHENSION

Definition: Short way to build lists.

Syntax:

```
[x*x for x in range(5)]
```

20. API REQUESTS

Definition: Fetch data from web APIs.

Syntax:

```
import requests
```

```
requests.get("https://api.com")
```